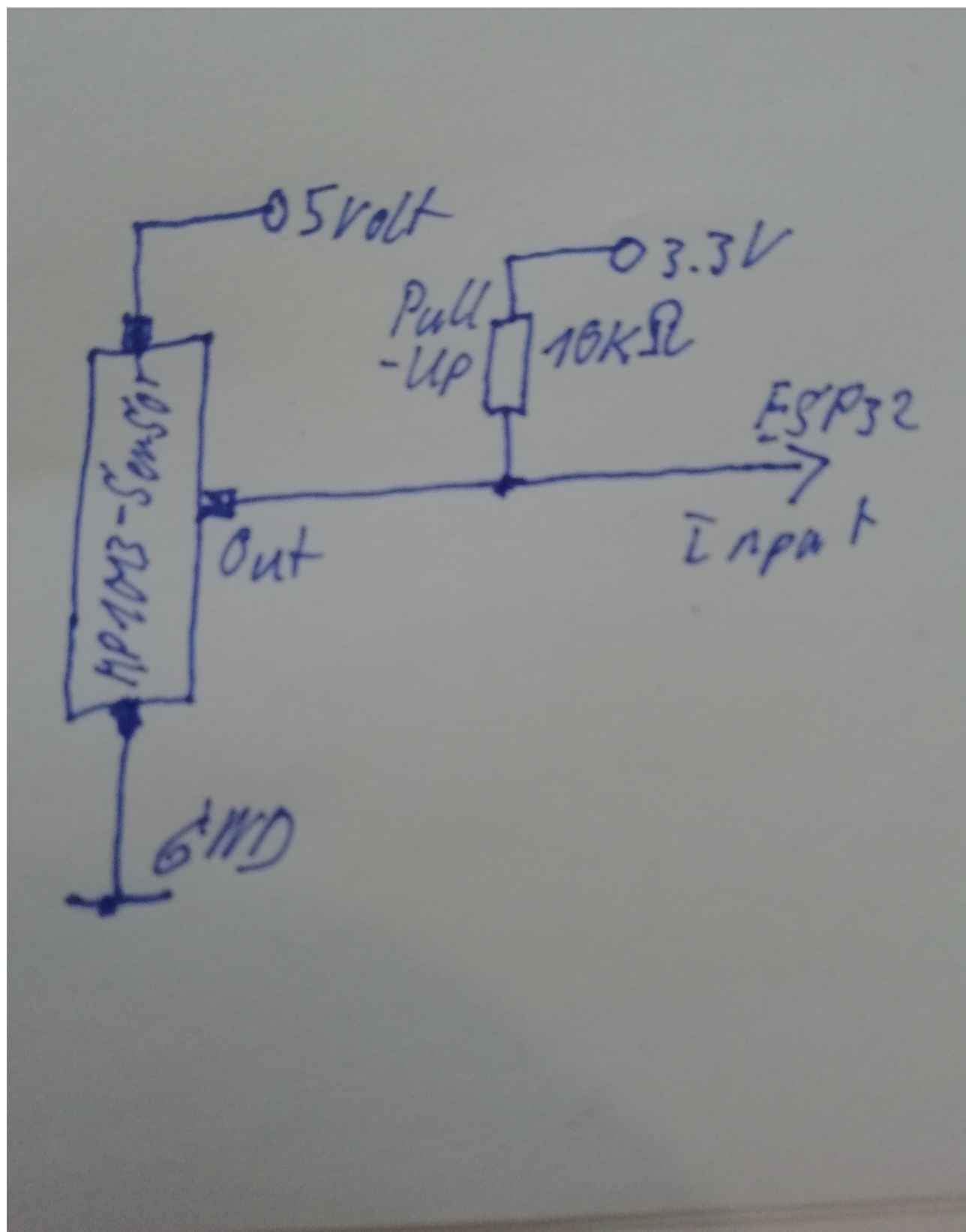


Das Vorhaben, einen **Hall-Sensor der MP1013-Serie** (von ZF / Cherry) an einem **ESP32** zur Geschwindigkeitsmessung am Fahrrad einzusetzen, lässt sich hervorragend umsetzen.

Es gibt dabei jedoch zwei kritische elektronische Besonderheiten, die beachtet werden müssen: Die Betriebsspannung des Sensors und sein Ausgangstyp.

Bei Conrad für 11 €uro zu bekommen.



1. Wichtige Hardware-Besonderheiten

-
- **Betriebsspannung:** Laut [MP1013-Datenblatt](#) benötigt der Sensor eine Versorgungsspannung von **4,75 V bis 24 V DC**. Er kann daher **nicht** mit den 3,3 V des ESP32 betrieben werden. Der Sensor muss an den **5V-Pin (V5 / Vin)** des ESP32-Boards angeschlossen werden. [3, 4]
- **Ausgang (Open Collector Sinking):** Der Sensor schaltet im aktiven Zustand den Ausgang gegen Masse (GND). Wenn der Magnet vorbeifährt, zieht der Sensor die Leitung auf 0V. Wenn kein Magnet da ist, ist die Leitung "offen". [4, 5, 6]
- **Spannungsschutz für den ESP32:** Ein ESP32 verträgt an seinen GPIO-Eingängen **maximal 3,3 V**. Da der Sensor mit 5 V versorgt wird, darf der notwendige Pull-up-Resistor **auf gar keinen Fall mit 5 V** verbunden werden! Stattdessen wird die Signalleitung über einen **10 kΩ Widerstand mit 3,3 V** verbunden oder der interne Pull-up des ESP32 genutzt. Da der Open Collector nur gegen GND schaltet, liegen so nie mehr als 3,3 V am GPIO an. [4, 5]
- **Magnet-Ausrichtung:** Die MP1013-Sensoren sind **Südpol-sensitiv**. Der Speichenmagnet muss also mit der richtigen Seite (Südpol) am Sensor vorbeigeführt werden, damit er auslöst. [1, 6, 7]
-

2. Verkabelung (Pinbelegung)

Die MP1013-Serie besitzt üblicherweise 3 Adern (AWG 24 Drähte).

(Prüfe zur Sicherheit die Kabelfarbe deines spezifischen Modells im Datenblatt, meistens gilt: Braun = VCC, Blau = GND, Schwarz = OUT). [6]

Sensor-Leitung	Verbindung am ESP32	Hinweis
VCC	5V / Vin Pin	Benötigt mind. 4,75 V zum Arbeiten.
GND	GND Pin	Gemeinsame Masse.
OUT / Signal	GPIO Pin (z.B. GPIO 13)	Jeder interruptfähige GPIO ist möglich.

Wichtig: Verbinde einen **10 kΩ Widerstand** zwischen dem gewählten **GPIO Pin** und dem **3.3V Pin** des ESP32. Alternativ reicht oft auch der interne Pull-up im Code (INPUT_PULLUP), ein externer Widerstand ist bei längeren Kabeln am Fahrrad wegen der Störanfälligkeit aber deutlich stabiler. [5]

3. Programmcode (Arduino IDE)

Um bei hohen Geschwindigkeiten keinen Impuls zu verpassen, wird die Messung über einen **Hardware-Interrupt** gelöst. Der Code misst die Zeit zwischen zwei Durchgängen (ISR) und berechnet daraus die Geschwindigkeit. [4, 8, 9]

```
#include <Arduino.h>
```

```
// Definition des Sensor-Pins  
const byte SENSOR_PIN = 13;
```

```
// Radumfang in Metern (Beispiel für 28 Zoll / 700x38C ca. 2.180 mm)  
// Passe diesen Wert exakt an dein Fahrrad an!  
const float RADUMFANG = 2.18;
```

```

// Variablen für die Zeitmessung (müssen 'volatile' sein, da in ISR genutzt)
volatile unsigned long letzteZeit = 0;
volatile unsigned long zeitDifferenz = 0;
volatile bool neuerImpuls = false;

// Entprell-Zeit in Millisekunden (verhindert Fehltrigger durch Signalrauschen)
const unsigned long ENTPRELL_ZEIT = 20;

// Interrupt-Service-Routine (ISR)
void IRAM_ATTR magnetImpuls() {
    unsigned long aktuelleZeit = millis();

    // Prüfen, ob die Entprellzeit abgelaufen ist
    if ((aktuelleZeit - letzteZeit) > ENTPRELL_ZEIT) {
        zeitDifferenz = aktuelleZeit - letzteZeit;
        letzteZeit = aktuelleZeit;
        neuerImpuls = true;
    }
}

void setup() {
    Serial.begin(115200);

    // Pin als Eingang mit Pull-up konfigurieren
    pinMode(SENSOR_PIN, INPUT_PULLUP);

    // Interrupt bei fallender Flanke aktivieren (Sensor zieht gegen GND)
    attachInterrupt(digitalPinToInterrupt(SENSOR_PIN), magnetImpuls, FALLING);

    Serial.println("Fahrradtacho gestartet. Warte auf Impulse...");
}

void loop() {
    // Wenn ein neuer Impuls registriert wurde
    if (neuerImpuls) {
        neuerImpuls = false; // Flag zurücksetzen

        // Geschwindigkeit berechnen
        // Formel: Umfang (m) / Zeit (ms) * 1000 = m/s
        // m/s * 3.6 = km/h
        float geschwindigkeit_ms = RADUMFANG / (zeitDifferenz / 1000.0);
        float geschwindigkeit_kmh = geschwindigkeit_ms * 3.6;

        Serial.print("Zeit pro Umdrehung: ");
        Serial.print(zeitDifferenz);
        Serial.print(" ms | Geschwindigkeit: ");
        Serial.print(geschwindigkeit_kmh, 1);
        Serial.println(" km/h");
    }

    // Timeout-Erkennung: Wenn sich das Rad länger als 2,5 Sekunden nicht dreht (Stillstand)
    if (millis() - letzteZeit > 2500 && letzteZeit > 0) {
        Serial.println("Stillstand... 0.0 km/h");
        letzteZeit = 0; // Verhindert dauerhaftes Feuern der Meldung
    }
}

```

4. Praxistipps für die Montage am Fahrrad

-
- **Kabelabschirmung:** Da die Leitungen vom Sensor an der Gabel bis zum Lenker (wo meist der ESP32 sitzt) recht lang sind, fangen sie leicht elektromagnetische Störungen ein. Verwende am besten ein abgeschirmtes Kabel oder verdrehe die Signal- und Ma
- Abstand:** Der Hall-Sensor muss sehr nah am Magneten vorbeigeführt werden. Der Abstand zwischen dem MP1013 an der Gabel und dem Speichenmagneten sollte **maximal 3 bis 5 mm** betragen. [4, 10]